



Student Project Collaboration Platform Complete Technical Documentation

Version 1.0 • August 2026

Spring Boot 3.3

Java 17

PostgreSQL

Next.js 14

React 18

TypeScript

Tailwind CSS

Table of Contents

- | | |
|--|---|
| 1. What is SIDEQUEST? | 8. API Reference |
| 2. How It Works (User Journey) | 9. Business Rules & Application Lifecycle |
| 3. System Architecture | 10. Frontend — Pages & Features |
| 4. Tech Stack | 11. Configuration Guide |
| 5. Database Design | 12. Running the Project Locally |
| 6. Backend — Structure | 13. Recent Security Improvements |
| 7. Backend — Security & Authentication | 14. Known Limitations & Roadmap |

1. What is SIDEQUEST?

SIDEQUEST is a web platform where university students can find teammates and build projects together. It wraps this collaboration in a fun, game-like theme:

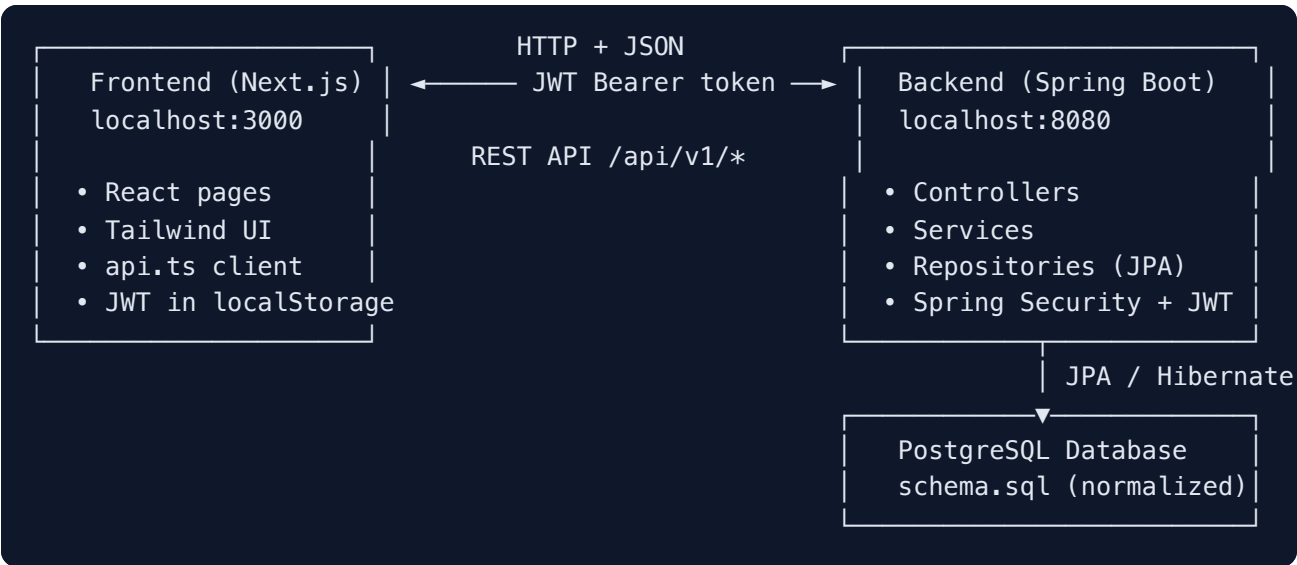
- **Quests** = projects that need teammates
- **Guilds** = communities/groups of students
- **Skill Trees** = hierarchical skill categories students can level up in
- **Badges & Achievements** = visual rewards shown on profiles
- **Player Cards** = student profile cards styled like RPG character cards

A student posts a project ("quest"), defines open team roles with the skills required for each role, and other students browse the quest feed and apply to the roles that match their skills. Project owners review applications and accept or reject them.

2. How It Works (User Journey)

1. **Sign up** — a student registers with email, password, full name, major, and college year.
2. **Build profile** — they add skills from the skill tree and pick a rank tier (Bronze → Silver → Gold → Platinum → Diamond).
3. **Browse quests** — the home feed shows open projects. Students can filter by skill, search by keyword, or filter by project status.
4. **Create a quest** — a student can post their own project, describing it and listing open roles (e.g., "Frontend Developer × 2, needs React").
5. **Apply** — another student applies to an open role. One application per role per person.
6. **Review** — the project owner sees all applicants (with their skills and profiles) and accepts or rejects each one.
7. **Team up** — accepted applicants become team members. When every spot across all roles is filled, the project automatically moves from OPEN to IN_PROGRESS.

3. System Architecture



Key architectural decisions

Decision	Why
Stateless JWT authentication	No server-side sessions; scales horizontally; simple for SPA clients
Layered backend (Controller → Service → Repository)	Clear separation of concerns; business logic lives only in services
DTOs for all requests/responses	Database entities are never exposed directly to API clients
Normalized relational schema	Data integrity enforced at the database level with constraints and foreign keys
<code>open-in-view: false</code>	Forces explicit data fetching; avoids hidden lazy-loading bugs

4. Tech Stack

Backend

Technology	Purpose
Java 17 + Spring Boot 3.3.5	Application framework and runtime
Spring Web	REST API endpoints
Spring Data JPA + Hibernate 6.5	Database access and object-relational mapping
Spring Security	Authentication and authorization
JJWT 0.12.6	Creating and validating JSON Web Tokens
Bean Validation (Jakarta)	Request input validation
Lombok	Removes boilerplate code (getters, builders, constructors)
PostgreSQL (prod) / H2 (dev & tests)	Databases
Maven	Build tool and dependency manager

Frontend

Technology	Purpose
Next.js 14 (App Router)	React framework with file-based routing
React 18	User interface library
TypeScript 5.5	Type safety
Tailwind CSS 3.4	Utility-first styling
Lucide React	Icon library

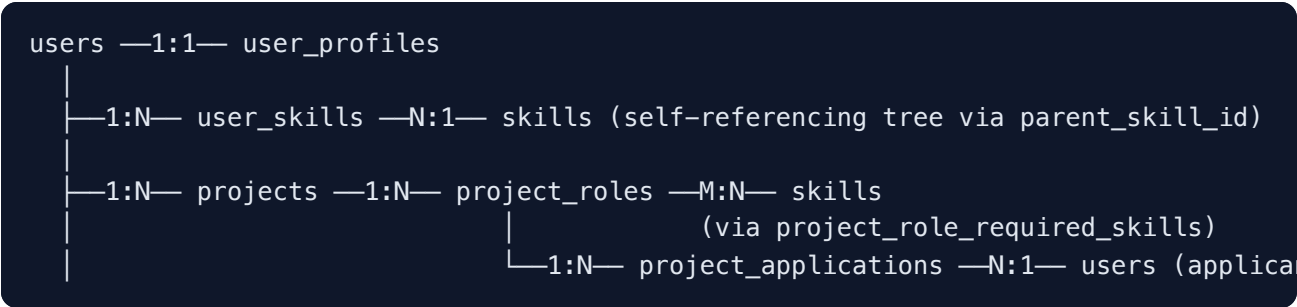
Database

Technology	Purpose
PostgreSQL	Main production database
CITEXT extension	Case-insensitive email storage
pgcrypto extension	Random UUID generation (<code>gen_random_uuid()</code>)

5. Database Design

The schema lives in `database/schema.sql`. All tables use UUID primary keys and have automatic `created_at` / `updated_at` timestamps (updated via a database trigger).

Entity Relationship Overview



Tables

users — login identity

Column	Type	Notes
id	UUID (PK)	Auto-generated
email	CITEXT (unique)	Case-insensitive
password_hash	TEXT	BCrypt hash — plain passwords are never stored

user_profiles — extended student info (one-to-one with users)

Column	Type	Notes
user_id	UUID (PK, FK → users)	Cascade delete
full_name	TEXT	Cannot be empty
college_year	SMALLINT	Must be between 1 and 8
major	TEXT	Cannot be empty
active_status	enum	ACTIVE / INACTIVE / SUSPENDED (default ACTIVE)

skills — hierarchical skill taxonomy

Column	Type	Notes
id	UUID (PK)	
skill_name	TEXT	Unique within its category
category	TEXT	e.g., "Programming Languages"
parent_skill_id	UUID (FK → skills)	Creates the tree structure; cannot be its own parent

user_skills — a user's proficiency in one skill

Column	Type	Notes
user_id + skill_id	unique pair	A user can claim each skill only once
rank_tier	enum	BRONZE / SILVER / GOLD / PLATINUM / DIAMOND
verification_status	enum	UNVERIFIED / PENDING / VERIFIED / REJECTED

projects — collaboration postings

Column	Type	Notes
owner_id	UUID (FK → users)	The creator; cannot be deleted while projects exist
title, description	TEXT	Required
repo_link	TEXT (optional)	Link to a Git repository
status	enum	DRAFT / OPEN / IN_PROGRESS / COMPLETED / ARCHIVED / CANCELLED

project_roles — open positions in a project

Column	Type	Notes
project_id	UUID (FK → projects)	Cascade delete
role_title	TEXT	e.g., "Backend Developer"
spot_count	SMALLINT	Must be greater than 0

project_role_required_skills — M:N join table

Links each role to the skills applicants should have.

project_applications — role applications

Column	Type	Notes
project_role_id + applicant_id	unique pair	One application per role per student
application_status	enum	PENDING / ACCEPTED / REJECTED (default PENDING)
applied_at	TIMESTAMPTZ	Indexed for sorting

6. Backend — Structure

```
backend/src/main/java/com/sidequest/
├── SidequestApplication.java      # Entry point (@SpringBootApplication)
├── config/
│   ├── SecurityConfig.java       # Filter chain, CORS, password encoder
│   └── JwtProperties.java         # JWT settings (secret, expiry) with validation
├── controller/                  # REST endpoints (thin – no business logic)
│   ├── AuthController.java       # /api/v1/auth/**
│   ├── ProfileController.java    # /api/v1/profiles/**
│   ├── SkillController.java      # /api/v1/skills/**
│   ├── ProjectController.java    # /api/v1/projects**
│   ├── ProjectApplicationController.java
│   └── HomeController.java
├── service/                     # All business logic lives here
│   ├── AuthService.java          # Register + login
│   ├── ProfileService.java
│   ├── SkillService.java         # Skill tree, user skill upsert
│   ├── ProjectService.java       # Create/search projects
│   └── ProjectApplicationService.java # Apply, accept/reject, withdraw
├── repository/                  # Spring Data JPA interfaces
├── entity/                      # JPA database entities
├── dto/
│   ├── request/                 # Incoming payloads (validated)
│   └── response/                # Outgoing payloads
├── enums/                      # Java enums mirroring DB enum types
├── exception/                   # Global error handling
└── security/                    # JWT filter, token provider, current-user hel
```

Error handling

- `ResourceNotFoundException` → HTTP 404 when something doesn't exist
- `UnauthorizedAccessException` → HTTP 403 when access isn't allowed
- `IllegalArgumentException` → HTTP 400 for bad business-rule requests
- All errors are converted to consistent JSON responses by `GlobalExceptionHandler`

7. Backend — Security & Authentication

Registration & Login Flow

1. User submits email + password (register also includes name, major, year).
2. Passwords are hashed with **BCrypt** before storage — never saved as plain text.
3. Emails are normalized to lowercase; duplicates are rejected.
4. On successful login, Spring's `AuthenticationManager` verifies credentials.
5. The server issues a signed **JWT token** valid for 24 hours containing the user's ID and email.

How protected endpoints work

1. Every request passes through `JwtAuthenticationFilter`.
2. The filter reads the `Authorization: Bearer <token>` header.
3. The token signature is verified against the server's secret key.
4. If valid, the user is authenticated for that request; otherwise the endpoint returns 401/403.
5. `CurrentUserService` gives services easy access to "who is logged in".

What is public vs. protected

Access	Endpoints
Public (no token)	Register, login, GET projects, GET skills
Authenticated (token required)	Profile read/update, create project, apply, manage applications, add skills
Dev-only	H2 console (only active with the <code>dev</code> profile)

CORS: the API only accepts browser requests from whitelisted origins (default: `http://localhost:3000`). Change it with the `CORS_ALLOWED_ORIGINS` environment variable.

8. API Reference

Base URL: `http://localhost:8080/api/v1` — all bodies are JSON.

Authentication

Method & Path	Auth	Description
POST <code>/auth/register</code>	Public	Create account. Body: <code>{ email, password, fullName, major, collegeYear }</code> . Returns token + user info (HTTP 201).
POST <code>/auth/login</code>	Public	Body: <code>{ email, password }</code> . Returns token + user info.

Profiles

Method & Path	Auth	Description
GET <code>/profiles/me</code>	Token	Get my profile including my skills
PUT <code>/profiles/me</code>	Token	Update my name, major, college year

Skills

Method & Path	Auth	Description
GET <code>/skills</code>	Public	Get the full skill tree (parents with children)
POST <code>/skills/me</code>	Token	Add/update one of my skills. Body: <code>{ skillId, rankTier }</code>

Projects

Method & Path	Auth	Description
POST <code>/projects</code>	Token	Create a project with roles and required skills
GET <code>/projects</code>	Public	List projects. Optional filters: <code>?skill=</code> , <code>?search=</code> , <code>?status=</code>
GET <code>/projects/{id}</code>	Public	Get one project with roles, team members, and spot counts

Applications

Method & Path	Auth	Description
POST /projects/{projectId}/roles/{roleId}/apply	Token	Apply to a specific role
PUT /applications/{id}/status	Token	Owner accepts/rejects; applicant may withdraw (reject own pending app)
GET /projects/{projectId}/applications	Token	Owner views all applicants for their project
GET /applications/me	Token	View my own sent applications

Example: register and call a protected endpoint

```
# 1. Register
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"alex@uni.edu","password":"secret123",
      "fullName":"Alex Rivera","major":"Computer Science","collegeYear":2}'

# Response: {"accessToken":"eyJhbG...", "tokenType":"Bearer", ...}

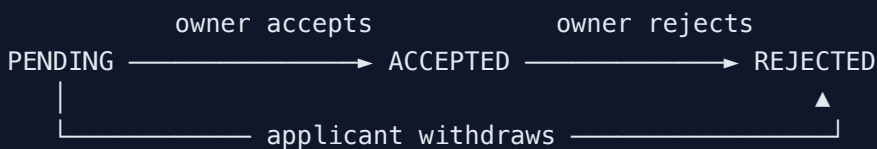
# 2. Call a protected endpoint
curl http://localhost:8080/api/v1/profiles/me \
  -H "Authorization: Bearer eyJhbG..."
```

9. Business Rules & Application Lifecycle

Rules enforced when applying to a role

- The project must be in **OPEN** status
- You **cannot apply to your own project**
- You can apply to each role **only once**
- The role must have **spots remaining**

Application state machine



Concurrency safety

When someone applies or an owner decides on an application, the system places a **pessimistic database lock** on the role row before counting accepted members. This means two people applying at the exact same moment can never over-fill a role — one waits briefly for the other to finish.

Automatic project status changes

Event	Effect on project status
Last available spot is accepted	OPEN → IN_PROGRESS (only when ALL roles are fully staffed)
An accepted member is rejected/withdrawn later	IN_PROGRESS → OPEN (others can apply again)

Data integrity guarantees

- Passwords stored only as BCrypt hashes
- Emails are unique and case-insensitive
- Deleting a user removes their profile, skills, and applications (cascade)

- A project owner cannot be deleted while their projects exist (restrict)
- Skills referenced by roles or users cannot be accidentally deleted (restrict)

10. Frontend — Pages & Features

```
frontend/src/
├── app/                # Pages (Next.js App Router)
│   ├── page.tsx        # Home — quest feed with search & filters
│   ├── quests/page.tsx # Browse all quests
│   ├── my-quests/page.tsx # My posted projects + received applications
│   ├── saved-quests/page.tsx # Bookmarked quests (localStorage)
│   ├── skills/page.tsx  # Interactive skill tree
│   ├── profile/page.tsx # My profile, badges, achievements
│   ├── guilds/[slug]/   # Guild (community) detail pages
│   └── communities/[slug]/ # Community detail pages
├── components/         # Reusable UI pieces (~35 components)
└── lib/
    ├── api.ts          # Typed REST client for the backend
    ├── savedQuests.ts  # Local bookmark persistence
    ├── theme-provider.tsx # Dark/light mode
    └── tierConfig.ts    # Rank-tier colors/config
```

Notable features

- **Quest cards** — show title, owner, required skills, open party roles, dates
- **Create Quest modal** — post a new project with multiple roles
- **Apply modal** — one-click application to a role
- **Skill node tree** — visual tree explorer; unlock/level skills
- **Badge system** — achievement grid, badge details, badge stats
- **Player profile card** — metal/RPG-styled card with tier customization
- **Dark/light theme toggle** — persisted preference
- **JWT session** — token kept in localStorage, auto-attached to API calls

Note: some sections of the home feed still use demo ("mock") quest data to make the prototype look alive. Real projects created through the API appear alongside them.

11. Configuration Guide

Environment variables (backend)

Variable	Required	Default	Purpose
<code>SPRING_PROFILES_ACTIVE</code>	No	<code>dev</code>	<code>dev</code> uses in-memory H2; any other value expects PostgreSQL
<code>DB_USERNAME</code> / <code>DB_PASSWORD</code>	In prod	<code>postgres</code>	PostgreSQL credentials
<code>JWT_SECRET</code>	Yes	—	Signing key, min 32 characters. App refuses to start without it.
<code>CORS_ALLOWED_ORIGINS</code>	No	<code>http://localhost:3000</code>	Browser origins allowed to call the API

Profiles

Profile	Database	Schema handling	Use
<code>dev</code> (default)	In-memory H2 (PostgreSQL mode)	Recreated on each restart	Local development
other (e.g., <code>prod</code>)	PostgreSQL at <code>localhost:5432/sidequest</code>	<code>validate</code> — must match <code>schema.sql</code>	Deployment

Important: without `JWT_SECRET` set, production startup fails immediately with a clear error message — this is intentional protection against silently running with a weak key.

12. Running the Project Locally

Prerequisites

- Java 17+, Node.js 18+, Maven (or use the included wrapper)
- PostgreSQL only if you want persistent data (dev mode needs nothing)

Step 1 — Start the backend (dev mode)

```
cd backend
./mvnw spring-boot:run
# Runs on http://localhost:8080 with in-memory H2 database
# H2 console: http://localhost:8080/h2-console
```

Step 2 — Run the database schema (optional, for PostgreSQL)

```
createdb sidequest
psql -d sidequest -f ../database/schema.sql
```

Step 3 — Start the frontend

```
cd frontend
npm install
npm run dev
# Opens on http://localhost:3000
```

Step 4 — Run the backend tests

```
cd backend
./mvnw test
```

13. Recent Security Improvements

The following hardening changes were applied during review (August 2026):

1. **CORS locked down** — was open to all origins with credentials; now restricted to whitelisted origins (default localhost:3000 only).
2. **JWT secret fail-fast** — the app validates at startup that the secret exists and is ≥ 32 bytes; no more silent insecure default in production.
3. **H2 console isolated** — only reachable under the dev profile.
4. **Race condition fixed** — pessimistic row locks prevent over-filling roles under concurrent applications/decisions.
5. **Smarter project status** — projects go IN_PROGRESS only when fully staffed, and reopen when a member leaves.
6. **Ownership check hardened** — viewing applicants now verifies ownership explicitly, even with zero applications.
7. **Schema drift prevented** — production uses `ddl-auto: validate` instead of auto-updating the schema.
8. **Typed API layer** — all frontend API responses have TypeScript interfaces instead of `any`.

14. Known Limitations & Roadmap

Current limitations

- No pagination yet — very large lists of projects/applications would slow down
- JWT tokens live 24 hours with no refresh-token flow or logout revocation
- Skill verification statuses exist in the data model but no verification process runs yet
- No notifications — applicants aren't told when they're accepted/rejected
- Token is stored in localStorage, which is readable by scripts if XSS ever occurred
- Some home-feed content is mock/demo data mixed with real data

Suggested next steps (in priority order)

1. **Flyway migrations** — version-controlled schema changes instead of manual SQL
2. **Refresh tokens + logout** — short-lived access tokens, revocable sessions
3. **Pagination** — for project listings and application lists
4. **Skill verification** — peer endorsement or GitHub-based proof of skills
5. **Notifications** — email or in-app alerts on application decisions
6. **More tests** — cover the application state machine and permission rules

